Intel SGX とブロックチェーンを用いた KGC に対して安全な ID ベース暗号の実装評価

宮 大介[1,a)]　北山 武輝[1]　佐藤 龍[2]　渡邉 英伸[3]　柿崎 淑郎[1]　大東 俊博[1,4]

概要：ID ベース暗号（IBE）には，鍵生成センタ（KGC）が任意の ID の秘密鍵を生成でき，全ての暗号文を復号可能となる鍵供託問題が存在する．これに対し北山らは，Intel SGX を用いてマスター秘密鍵と鍵発行処理を KGC から秘匿し，署名付き鍵生成ログをブロックチェーンに記録することで KGC による不正な鍵発行を抑止する方式を提案している．本稿では，この方式の鍵発行処理を実装し，その性能を評価する．本実装で使用するペアリング暗号ライブラリ mcl は標準では GMP に依存し，OS のシステムコールを利用できない SGX の Enclave 内では動作させることが困難である．そこで GMP を無効化して，mcl 付属の VInt class を用いるようにビルドすることで Enclave 内でのペアリング演算を実現している．さらに，Intel SGX ハードウェア上で北山らの方式の初期化処理および鍵発行処理に関わる実行時間を計測し，実用的な時間で実行可能であることを明らかにした．

キーワード：ID ベース暗号，鍵供託問題，Intel SGX，ブロックチェーン，ペアリング暗号

Implementation and Evaluation of Identity-Based Encryption with Security against the KGC Using Intel SGX and Blockchain

Daisuke Miya[1,a)]　Takeki Kitayama[1]　Ryu Sato[2]　Hidenobu Watanabe[3]　Yoshio Kakizaki[1]
Toshihiro Ohigashi[1,4]

Abstract: Identity-Based Encryption (IBE) has an inherent key escrow problem: the Key Generation Center (KGC) can generate the secret key for any ID and thus decrypt all ciphertexts. To address this, Kitayama *et al.* proposed a scheme that uses Intel SGX to hide the master secret key and the key generation process from the KGC, and records signed key-generation logs on a blockchain to deter illicit key generation by the KGC. In this paper, we implement and evaluate the key generation part of this scheme. The pairing-based cryptography library mcl used in this work depends on GMP by default and is difficult to run inside an SGX enclave, which cannot use OS system calls. Therefore, we disabled GMP and built mcl to use its bundled VInt class, enabling pairing operations inside the enclave. Furthermore, we measured the execution time related to the initialization and key generation of the scheme on Intel SGX hardware, and showed that it runs in a practical time.

Keywords: Identity-based encryption, Key escrow problem, Intel SGX, Blockchain, Pairing-based cryptography

1. はじめに

従来の暗号技術より高度な機能を提供する高機能暗号の一種として，メールアドレスや社員番号等の識別子（ID）

[1] 東海大学
Tokai University
[2] 情報セキュリティ大学院大学
Institute of Information Security
[3] 広島大学
Hiroshima University
[4] 国立研究開発法人情報通信研究機構
National Institute of Information and Communications Technology
[a)] 6cjnm035@tokai.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

を用いて暗号化が可能な ID ベース暗号（Identity-Based Encryption, IBE）[1], [2] が知られている．IBE では，あらかじめ全利用者にマスター公開鍵を安全に共有している前提のもとで ID から直接暗号化鍵を導出できるため，公開鍵基盤（Public Key Infrastructure, PKI）が提供するサーバ証明書のような，ドメイン等の ID と公開鍵を紐づける機能が不要となる．また, 従来の公開鍵暗号と異なり事前に暗号化鍵・復号鍵ペアを生成することが不要であるため．復号鍵を生成する前に暗号化処理を実行することも可能である．これらの従来の暗号技術には無い性質から secure e-mail[2] やセキュア通信 [3] などへの応用が進められている．

一方で，IBE では鍵生成センタ（KGC: Key Generation Center）が ID に対応する全ての秘密鍵を生成できることから，全ての暗号文を復号可能というプライバシー上の課題を抱えている．これを鍵供託問題と呼ぶ．この問題への対策として，鍵生成時に対象の ID を KGC に秘匿するブラインド署名を用いた手法 [3], [4], [5] などが提案されているが，暗号文に対応する ID を KGC に秘匿する必要があるなど，適用範囲に制約がある．

IBE の実用・普及を考えた場合，ID の秘匿の条件が不要かつ鍵供託問題を解決できる技術が必要である．北山らは，鍵供託問題における本質的な課題はマスター秘密鍵に KGC 管理者が直接アクセスできることであると考え，これを解決する技術として Intel SGX（Intel Software Guard Extensions）に着目し，マスター秘密鍵を KGC 管理者に秘匿した状態での鍵発行処理を実現するプロトコルを提案した [6]．さらに同方式では，署名付きの鍵生成ログをブロックチェーンに記録することで監査を可能とし，KGC による不正な鍵生成を抑止する仕組みを与えている．

ある方式が現実的に利用可能であるかを判断するには，実装による性能評価が不可欠である．本稿では北山らの方式の実装・評価を目的とし，北山らの方式ではペアリング演算を必要とするが，Intel SGX の SDK はペアリング演算を提供していないため対応が必要である. そこで本稿では，ペアリング暗号ライブラリの中でも高速な mcl[7] に注目し，Intel SGX の Enclave 上で動作する方法を調査した上で実装および評価を行う．なお，mcl は標準では多倍長整数演算に GMP を用いるが，GMP はシステムコールや独自のメモリ管理に依存するため，Enclave 内でそのまま動作させることは困難である．

本稿では，北山らの提案プロトコルのうち，特に Intel SGX に関わる鍵発行処理の実装と評価を行う．本稿の貢献は以下のとおりである．

- mcl を `MCL_USE_VINT` オプションを有効にしてビルドすることで GMP 依存を排除し，Enclave 内でのペアリング演算を実現した．
- 北山らの方式の初期化処理（Setup，シーリング）および鍵発行処理（Extract，ID に対応する秘密鍵の暗号化，暗号文および鍵生成ログへの署名，ログ生成）を Enclave 内に実装した．
- 実 SGX ハードウェア上で北山らの方式と従来の IBE の実行時間を計測し，鍵供託問題対策を付加した北山らの方式のオーバヘッドについて定量的に評価した．

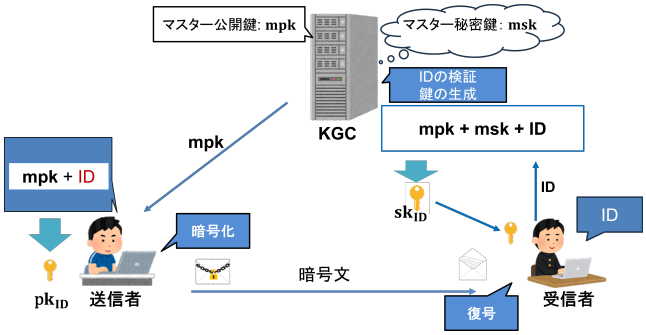


図 1 ID ベース暗号

2. 準備

2.1 ID ベース暗号

ID ベース暗号は公開鍵に任意の値を設定可能な公開鍵暗号の一種である [2]．KGC がマスター公開鍵 mpk とマスター秘密鍵 msk を管理し，msk を用いて任意の値（以下 ID と表記）に対する秘密鍵 sk_{ID} を生成し，sk_{ID} を ID を持つ利用者に配布する．なお，本稿では今後 sk_{ID} のことを「個別秘密鍵」と呼ぶ．暗号化時には mpk に加えて ID を指定し，対象の sk_{ID} を用いると暗号文が復号できる．ID ベース暗号の概要を**図 1** に示す．ここで示しているのは北山らの方式が採用している Boneh–Franklin の IBE[2] であるが，安全性および効率性の向上のため，非対称ペアリング版 [5] の構成を用いる．以下にその構成を示す．

まず，位数 q の三つの巡回群 $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ に対し，写像 $e\colon \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ が，双線形性，非退化性，および計算可能性を満たすとき双線形写像（ペアリング）と呼ぶ．本研究ではペアリングフレンドリ曲線 BLS12-381[8] を用いる．

Setup(1^λ)： セキュリティパラメータ λ を入力とし，素数 q を決める．q に従い楕円曲線群 $\mathbb{G}_1, \mathbb{G}_2$ とペアリング関数 $e\colon \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ を生成する．乱数 $g = \langle g_1 \in \mathbb{G}_1,\ g_2 \in \mathbb{G}_2 \rangle$ を生成する．乱数 $s \in \mathbb{Z}_r$ を生成し，$p = \langle p_1 = g_1^s,\ p_2 = g_2^s \rangle$ とする．暗号学的ハッシュ関数 $H\colon \{0,1\}^* \to \mathbb{G}_1^*$ を用意し，平文空間を $\mathcal{M} = \{0,1\}^n$，暗号空間を $\mathcal{C} = \mathbb{G}_1^* \times \{0,1\}^n$ とする．公開パラメータ $params = \langle q, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, n, e, g, p, H \rangle$ を出力する．マスター秘密鍵は s である．

Extract(ID, s)： 文字列 $ID \in \{0,1\}^*$ を受け取り，$u_{ID} =$

© 2026 Information Processing Society of Japan

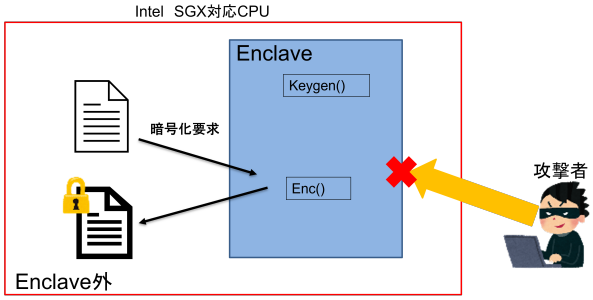


図 2 Intel SGX の概要

$H(ID) \in \mathbb{G}_1^*$ を計算する．マスター秘密鍵 s を用いて個別秘密鍵 $sk_{ID} = u_{ID}^s$ を出力する．

Encrypt(ID, p, g): $u_{ID} = H(ID)$ を計算する．$M \in \mathcal{M}$ を暗号化する平文とし，乱数 $r \in \mathbb{Z}_r$ を生成し，暗号文 $C = \langle c_0, c_1 \rangle = \langle g_1^r,\ M \oplus e(u_{ID}, p_2) \rangle$ を計算する．

Decrypt(C, sk_{ID}): 暗号文 $C = \langle c_0, c_1 \rangle$ と個別秘密鍵 sk_{ID} を用いて，$c_1 \oplus e(sk_{ID}, c_0) = M$ を計算する．

2.1.1 鍵供託問題

KGC は利用者の ID に対して個別秘密鍵 sk_{ID} を生成する．すなわち，鍵生成センタは利用者の ID と個別秘密鍵のペアを所持していることになる．そのため，鍵生成センタは暗号文を入手すると，それを不正に復号できるという強い権限を持っている．これは鍵供託問題と言われ，ID ベース暗号の課題として挙げられる．

2.2 Intel SGX

Intel SGX とは Intel が提案するハードウェアサポートによるセキュアな実行環境である（**図 2**）．Intel SGX は CPU 内部に保持している秘密鍵によって Enclave と呼ばれる暗号的に厳重に保護された領域を生成し，Enclave 内でデータを処理することでデータ保護を実現している．これにより，メモリ内部を覗き見るような攻撃やハードウェア自体への攻撃に対して強い耐性を持つ．一方で，実行時間計測やキャッシュヒット率の計測を利用したサイドチャネル攻撃に対しては脆弱性があると知られており [9], [10]，脆弱性を緩和するには Intel から提供されるパッチの適用や BIOS の更新を頻繁に行う必要がある．

2.2.1 Enclave

Enclave とは Intel SGX が生成する暗号的に厳重に保護された領域であり，Enclave 内のデータやコード（Enclave イメージ）は厳重に保護される．Enclave イメージはビルド時に作成者の署名鍵で署名され，実行時にその署名とハッシュ値が検証される．これにより，署名後に Enclave の内容が改ざんされた場合は検証に失敗し，実行が拒否されるため，Enclave の完全性と信頼性が確保される．

Intel SGX が信頼しているのはインテル製 CPU と Enclave 内の実行のみであるため，OS にはユーザモードの権限しか与えられていない．よって，Enclave 内ではシステムコールを使用できない制限の中で実装する必要がある．Enclave 内外を跨ぐ処理を行う場合は ECALL，OCALL と呼ばれる呼び出し関数を用いる．ECALL 関数は非保護領域から Enclave の処理を呼び出す際に使うものであり，OCALL 関数は Enclave から非保護領域の処理を呼び出す際に使用し，システムコールを利用したい場合に用いられる．これらのインタフェースは EDL（Enclave Definition Language）で定義する．

2.2.2 シーリング，アンシーリング

データを永続化するためには揮発性メモリから非揮発性メモリに情報を保存する必要がある．しかし，Enclave 内で処理される情報をそのまま非揮発性メモリに保存すると，Enclave 外部からのアクセスにより機密情報が漏洩するおそれがある．そこで Intel SGX はシーリング機能を提供している．シーリングとは，Enclave 内からしかアクセスできないハードウェア鍵から導出される秘密鍵を用いて暗号化することであり，この機能を用いることで Enclave 内のデータを安全に外部に保存できる．シーリングされた情報を Enclave 内にロードし復号する機能をアンシーリングと呼ぶ．シーリングで用いられた鍵は Enclave の固有情報を基に生成されるため，シーリングされたデータを別の Enclave では復号できない．本稿では，平文 M をシーリングした結果を Seal$(M) \rightarrow sealdata$，アンシーリング操作を Unseal$(sealdata) \rightarrow M$ と表す．

2.3 mcl ライブラリ

mcl[7] は 光成氏 によって開発されているペアリング暗号ライブラリであり，BN254 や BLS12-381 などの曲線に対応する．標準構成では多倍長整数演算に GMP を用いるが，mcl は内部に純粋な C++ 実装の多倍長整数演算 VInt class も備えており，ビルド時に `MCL_USE_VINT` を有効化することで GMP へ依存しない自己完結した構成にもできる．

2.4 ブロックチェーンと Ethereum

Ethereum は分散型アプリケーションを実行する環境を備えたオープンソースのブロックチェーンプラットフォームである．Ethereum の特徴にスマートコントラクトがある．スマートコントラクトとは，ブロックチェーン上で自動的に実行されるプログラムであり，契約条件の記述やトランザクション処理を人手を介さずに行うことができる．ブロックチェーンは改ざん耐性を有するため，記録の隠蔽や改ざんを防止できる．

© 2026 Information Processing Society of Japan

3. 北山らの方式

本節では，本稿が実装対象とする北山らの方式 [6] の概要を述べる．

3.1 概要

北山らの方式では，Intel SGX の Enclave 内でマスター秘密鍵（msk）と ID に対応する個別秘密鍵を生成する．これにより，KGC 管理者はマスター秘密鍵に直接アクセスできず，利用者の ID を知っていても個別秘密鍵を独自に生成することはできない．しかし，KGC が自身の公開鍵を用いて任意の個別秘密鍵 sk_{ID} を奪取する攻撃が考えられる．そこで本方式では，Enclave 内で鍵生成ログを作成する．ログには連続性を示すシーケンス番号を付与し，Enclave 内で生成された個別秘密鍵および対応するログに署名を行う．利用者は個別秘密鍵の署名の検証を行うことで，自身の個別秘密鍵 sk_{ID} が正しく Enclave 内で生成されたことを確認できる．さらに，生成ログとログの署名をブロックチェーン（Ethereum）上に記録することで，KGC による鍵発行の順序を追跡でき，KGC の不正行為を抑制できる．

以下では $(\mathrm{Sig.KeyGen}, \mathrm{Sig.Sign}, \mathrm{Sig.Verify})$ を署名アルゴリズムとする．

3.2 初期化処理のフロー

Enclave.Setup は Enclave に対して初回のデプロイを行った際に実行される（**図 3**）．はじめにマスター公開鍵 mpk，マスター秘密鍵 msk を生成する．続いて，検証鍵 vk，署名鍵 ik を生成する．この生成過程での乱数は Enclave 内で発生させる．これらの鍵生成が完了した後，Intel SGX のシーリング機能を用いて msk，ik をシーリングし，$\mathrm{Seal}(msk) \to S_{msk}$，$\mathrm{Seal}(ik) \to S_{ik}$ として KGC の内部ストレージなど Enclave 外部に保存する．またシーケンス番号 seq を生成し初期化を行い，$\mathrm{Seal}(seq) \to S_{seq}$ として同様に保存する．

3.3 個別秘密鍵発行処理のフロー

利用者が ID に対応する個別秘密鍵の発行を依頼するフローを**図 4** に示す．利用者は事前に，sk_{ID} を安全に受け渡すための暗号化用公開鍵 pk_{IDuser} と秘密鍵 sk_{IDuser} を生成し所持している．利用者はまず ID，pk_{IDuser} と，自身が ID の所有者であることを証明する認証情報を KGC に安全に送信する．KGC は受け取った ID に対して認証を行い，問題がなければ Enclave に鍵導出を依頼する．

Enclave 内では，まずアンシーリング操作 $\mathrm{Unseal}(S_{msk}) \to msk$，$\mathrm{Unseal}(S_{ik}) \to ik$，$\mathrm{Unseal}(S_{seq}) \to seq$ を実行する．続いて以下の処理を行う．

表 1 実装環境

項目	内容
CPU	Intel Core i3-7100U @ 2.40GHz
メモリ	4 GB
OS	Ubuntu 16.04.6 LTS
SGX SDK / PSW	2.5.101.50123
動作モード	HW モード
OpenSSL	1.0.2s
ペアリングライブラリ	mcl (BLS12-381, `MCL_USE_VINT=1`)

IBE.Extract: ID と msk を入力し，ID に対する個別秘密鍵 sk_{ID} を生成する．

Enclave.Enc: sk_{ID} を利用者の公開鍵 pk_{IDuser} で暗号化し暗号文 C を生成する．その後，$\mathrm{Sig.Sign}(ik, C) \to Sig_C$ を実行し，暗号文 C に署名する．

Enclave.logGen: シーケンス番号を $seq = seq + 1$ と更新し，$(\mathrm{Hash}(ID), seq, pk_{IDuser}, C)$ のタプルを鍵生成情報とする LOG を生成する．ハッシュ関数を用いて $H_{LOG} = \mathrm{Hash}(LOG)$ を生成し，$\mathrm{Sig.Sign}(ik, H_{LOG}) \to Sig_{LOG}$ を実行してログに署名する．更新した seq に対し $\mathrm{Seal}(seq) \to S_{seq}$ を行う．

KGC は C, Sig_C を利用者に，LOG, Sig_{LOG} をブロックチェーンに送信し書き込む．利用者は C, Sig_C を受け取り $\mathrm{Sig.Verify}(vk, Sig_C)$ を実行し，検証に成功した場合に C を復号して sk_{ID} を得る．また，ブロックチェーンから LOG, Sig_{LOG} を取得して検証することで，ログの正当性を確認できる．

4. 実装

本節では，北山らの方式の Enclave 内処理に関する実装について述べる．実装環境を**表 1** に示す．なお，本稿ではライブラリやサンプルコードの豊富さから Intel SGX を前提にしているが，Remote Attestation などの機能を用いていないことから，他の TEE (Trusted Execution Environment) でも同様の実装が可能と考えている．

4.1 Enclave 内での mcl の実装

第 2.3 節で述べたとおり，mcl は多倍長整数演算を GMP から VInt class へ切り替えることができる．本実装の中心となる課題は，GMP に依存する mcl を Enclave 内で動作させることである．Enclave はシステムコールや標準ライブラリの一部を利用できないため，GMP をそのままリンクすることはできない．そこで，mcl を `MCL_USE_VINT=1` を指定して静的ライブラリとしてビルドし，多倍長整数演算を mcl 内部の VInt class に置き換えることで GMP 依存を排除した．加えて，Enclave 内で利用できない外部依存を

© 2026 Information Processing Society of Japan

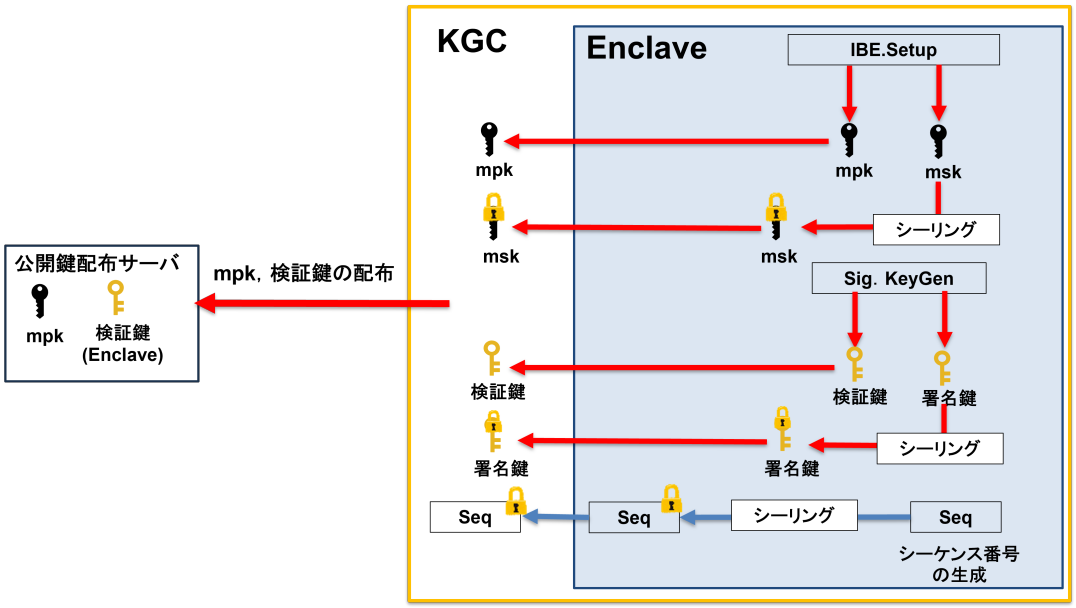


図 3　初期化処理のフロー

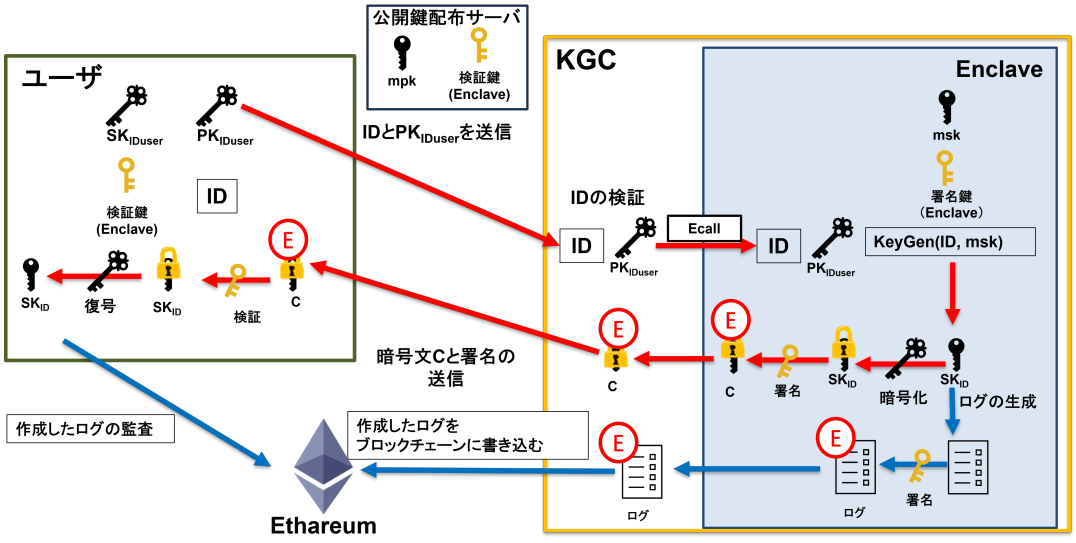


図 4　個別秘密鍵発行処理のフロー

取り除き，必要となる標準 C ライブラリ関数は SGX SDK の tlibc により提供した．これらの対応により，$\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ 上の群演算およびペアリング e をすべて Enclave 内で実行できるようにした．

また，ペアリング演算に用いる曲線として BLS12-381 を指定して mcl を初期化する．mcl は標準では乱数生成に OS の乱数源を用いるが，Enclave 内ではこれを利用できない．そこで `MCL_DONT_USE_CSPRNG` を指定して mcl 標準の乱数生成を無効化し，乱数の生成には SGX が提供する `sgx_read_rand` を用いるようにした．

ビルドした mcl は静的ライブラリ（`libmcl.a`）として Enclave にリンクする．

4.2　各処理の実装

北山らの方式の初期化処理および鍵発行処理を，それぞれ ECALL として Enclave 内に実装した．初期化処理で

© 2026 Information Processing Society of Japan

は，マスター鍵生成（Setup）と，msk, ik, seq のシーリングを行う．鍵発行処理では，シーリングされた鍵のアンシーリングに続き，Extract による sk_{ID} の生成，sk_{ID} の暗号化，暗号文 C への署名，鍵生成ログ LOG の生成，および LOG への署名を行う．

個別秘密鍵 sk_{ID} の配布には楕円エルガマル暗号を用いる．利用者はあらかじめ鍵ペア $(x,\ Y = xG)$ を持つものとし（G は生成元，x は秘密鍵），Enclave は sk_{ID}（$\mathbb{G}_1$ の元）と乱数 $k \in \mathbb{Z}_r$ を用いて $C_1 = kG$，$C_2 = sk_{ID} + kY$ として暗号化し，(C_1, C_2) を出力する．利用者は自身の秘密鍵 x を用いて $C_2 - xC_1 = sk_{ID}$ により復元する．これにより，鍵配布の過程で KGC 管理者が sk_{ID} を平文で読み取ることを防ぐ．暗号文 C および鍵生成ログのハッシュ値 H_{LOG} に対する署名には ECDSA を用いる．署名に用いる楕円曲線は secp256r1（P-256）であり，Enclave 内では SGX 暗号ライブラリ（`sgx_ecdsa_sign`）を，比較のための Enclave 外の実装では OpenSSL（曲線として prime256v1 を指定）を用いた．なお，IBE のペアリング演算に用いる BLS12-381 は署名用の曲線とは別である．また，利用者の公開鍵など Enclave 外の OpenSSL から受け取るデータはビッグエンディアンで表現されるため，mcl や SGX が用いるリトルエンディアンへ変換して扱う．マスター秘密鍵 msk および署名鍵 ik は，シーリングによって暗号化した状態でのみ Enclave 外部に保存する．

5. 評価

5.1 計測方法

北山らの方式の各処理の実行時間を計測する．しかし Enclave 内では `gettimeofday` などの時間取得関数を直接呼び出すことができない．これらの関数はシステムコールを必要とし，SGX の trusted runtime からは直接利用できないためである．そこで本稿では，非保護領域側で現在時刻を返す OCALL（`ocall_gettimeofday`）を用意し，計測対象の処理区間の前後で Enclave からこれを呼び出して開始・終了時刻を取得し，その差を処理時間とする．ECALL の入口コストは測定区間外とするが，OCALL の往復コストは測定区間に含まれる．

さらに，Enclave 化に伴うオーバヘッドを定量的に示すため，同一の処理を Enclave 外（非保護領域）でも実行して比較する．Enclave 外では，暗号処理に OpenSSL，ペアリング演算に mcl のホスト版ライブラリを用いる．各処理は 100 回計測し，その平均を求めるとともに，Enclave 内の平均を Enclave 外の平均で割った倍率（内/外）を算出する．なお，従来の IBE で用いていない処理については Enclave 内のみの計測とする．

5.2 計測項目

計測対象を**表 2** に示す．北山らの方式の処理フローに対応させ，初期化処理として Setup とシーリングを，鍵発行処理として Extract，sk_{ID} の楕円エルガマル暗号化，暗号文 C への署名，LOG 生成，LOG への署名を計測する．北山らの方式の鍵発行処理は，従来の IBE の Extract に加えて，KGC による不正を防ぐための sk_{ID} の暗号化や署名・ログの生成から構成される．そのため，鍵発行処理に要する時間はこれらの処理の合計として評価する．IBE のメッセージ暗号化（Encrypt/Decrypt）は KGC の鍵発行処理に含まれないため計測対象から除外し，署名検証（Verify）も KGC 側ではなく利用者側の処理であるため対象外とする．なお比較のために，鍵供託問題対策の機能を持っていない従来の IBE を Enclave 外で mcl により実装し，機能追加によるオーバヘッドを評価する．具体的には表 2 でチェックを入れている処理を Enclave 外で計測して比較に用いる．

表 2 北山らの手法の計測項目

#	処理	処理内容	従来の IBE でも実行される処理
	初期化処理		
1	Setup	s, $P_{\mathrm{pub}} = sP$ の生成	✓
2	シーリング	msk・ik・seq の封印	
	鍵発行処理		
3	Extract	$sk_{ID} = sH(ID)$ の生成	✓
4	sk_{ID} 暗号化	楕円エルガマル暗号化	
5	C への署名	Sig_C の生成	
6	LOG 生成	鍵生成ログの生成	
7	LOG への署名	Sig_{LOG} の生成	

表 3 各処理の実行時間（100 回平均）．内・外は Enclave 内・Enclave 外，倍率は内/外を表す．「—」は Enclave 外に対応する処理が無いことを示す．

処理	内 [ms]	外 [ms]	倍率
Setup	0.4488	0.1233	3.64
シーリング	0.0416	—	—
Extract	2.1296	0.4914	4.33
sk_{ID} 暗号化	0.4414	—	—
C への署名	0.3318	—	—
LOG 生成	0.0088	—	—
LOG への署名	0.3292	—	—

5.3 計測結果

各処理の実行時間を**表 3** に，初期化処理および鍵発行処理に要する時間の合計を**表 4** に示す．

表 4 より，初期化処理に要する時間は，従来の IBE では約 0.12 ms であるのに対し，北山らの方式では約 0.49 ms であり，約 3.98 倍となった．表 3 に示すとおり，北山らの方式の初期化処理は Setup に加えてマスタ鍵等のシーリングを含むが，これらは Enclave のデプロイ時に一度だけ

© 2026 Information Processing Society of Japan

表 4 北山らの方式と従来の IBE の初期化処理・鍵発行処理時間の比較（単位 ms）.

処理	北山らの方式	従来の IBE	倍率
初期化処理	0.4904	0.1233	3.98
鍵発行処理	3.2408	0.4914	6.60

実行される処理である.

鍵発行処理に要する時間は，従来の IBE では Extract のみの約 0.49 ms であるのに対し，北山らの方式では約 3.24 ms であり, 約 6.60 倍となった．表 3 より, 北山らの方式の鍵発行処理を構成する各処理のうち, Extract の実行時間が Enclave 内で約 2.13 ms と最も大きく，鍵発行処理全体の約 66% を占めた．これは, Extract が $sk_{ID} = H(ID)^s$ という $\mathbb{G}_1$ 上のべき乗算を含み，群の加算のみで構成される処理や 32 バイトのハッシュ値に対する署名処理と比べて計算量が大きいためと考えられる．一方，LOG 生成はタプルの構成とハッシュ計算のみであるため，約 0.009 ms と無視できる程度に小さい.

表 3 より, 従来の IBE の KGC でも実行される Setup と Extract という個別の関数の処理時間について Enclave 内外を比較すると，Enclave 内の実行時間は Enclave 外のそれぞれ約 3.6 倍，約 4.3 倍であった．この差には，Enclave の実行そのものに伴うオーバヘッドに加えて，Enclave 外では GMP を用いる mcl のホスト版を，Enclave 内では GMP に依存しない VInt class を用いることによる多倍長整数演算の性能差が含まれる．とくにペアリング群上の演算が支配的な Extract は，この VInt class と GMP の性能差の影響を強く受けていると考えられる.

IBE における鍵発行処理は利用者の利用開始時や鍵の再発行時にのみ行われるものであり，通信のたびに実行されるものではない．したがって，北山らの方式の鍵発行処理全体の実行時間が Enclave 内で約 3.24 ms であることは，鍵発行の頻度を踏まえれば実用上許容できる範囲であると考えられる．なお, 実運用における鍵発行の応答時間には, 本稿の評価対象に含まれない Remote Attestation，利用者と KGC 間の通信・認証，ブロックチェーンへの書き込み，利用者側での復号や署名検証などの処理も加わる.

6. 考察

本節では，文献 [6] で議論された北山らの方式の安全性について整理し，高い安全性を確保するためにはプロトコルの修正が必要になることを示す．本検討は，定められたプロトコルを正しく実行する条件において得られた情報から秘密情報（個別秘密鍵）を得ようとする semi-honest モデルと，攻撃者に対して秘密情報を得るためにプロトコルから逸脱した行動を許す malicious モデルについて検討する．malicious モデルに対して安全であれば高度な安全性を有していると考えられる.

6.1 semi-honest モデルにおける安全性

semi-honest モデルでは，KGC は鍵発行プログラムを改ざんせず，正規の処理の中で秘密情報を得ようとする．北山らの方式では，マスター秘密鍵と鍵発行処理は Enclave 内に秘匿されるため，KGC がマスター秘密鍵を使って任意の ID に対応する個別秘密鍵を生成することを防ぐことができる．また，Enclave から個別秘密鍵を出力する際，ID を有しているユーザから送信された公開鍵で暗号化するため，KGC 管理者は暗号文から個別秘密鍵を得ることができない．KGC による公開鍵のすり替えについては鍵生成ログの中に ID に対応する鍵送付用の公開鍵を含めておき，ブロックチェーン上で公開されたものを確認することで検出できる.

次に，KGC が一般の利用者を装い，プロトコルに則って鍵発行プログラム実行することで任意の ID に対する個別秘密鍵の発行を試みる攻撃を考える．現在のプロトコルでは KGC が ID に対応する認証をして鍵発行処理の実行可否を決定することから，KGC が他人の ID に対して許可を与えた上で任意の公開鍵と秘密鍵を使って鍵発行処理を実行することは semi-honest モデルの範疇と考えている．KGC が鍵発行プログラムを実行した場合，個別秘密鍵に対応する ID はブロックチェーン上のログから確認できるため，不正な鍵生成は検出できる.

6.2 malicious モデルにおける安全性

malicious モデルでは，攻撃者はプロトコルから逸脱した行動をとる．本稿ではブロックチェーンへの鍵生成ログの書き込みおよびシーケンス番号のシーリングデータの改ざんに注目して議論する.

まず, Enclave が発行した鍵生成ログの一部を KGC がブロックチェーンに書き込まないことで隠蔽する攻撃を考える．これに対しては，Enclave が出力する署名付きの鍵生成ログにはシーケンス番号が含まれていることから，シーケンス番号の連続性を確認することで KGC の不正を検出できる．ただし，新規の鍵発行リクエストが頻繁に生じない場合はシーケンス番号が連続していないことを検出されずに発覚が遅くなることから，文献 [6] では KGC サービスの運用主体が定期的にダミーの ID を用いて鍵発行リクエストを送り，最新のシーケンス番号が含まれているログを確認できるようにする対応が議論されている.

次に，シーリングされたシーケンス番号（暗号文）を過去のものに置き換える攻撃が想定される．KGC はシーリングされたデータを自由に作ることはできないが，過去のものに置き換えることで古い正規のデータとして Enclave に読み込ませることができる．これはリプレイ攻撃の一種である．この方法によって鍵発行時に使われるシーケンス番号は古いものに巻き戻されることになり，不正に鍵発行プログラムを実行した上でシーケンスを 1 つ巻き戻すこと

© 2026 Information Processing Society of Japan

で個別秘密鍵の生成を隠蔽することが可能となる．

このような脆弱性への対策として，KGC 管理者が過去の情報に巻き戻すことができないようにプロトコルを修正する必要がある．具体的には，TPM の単調増加カウンタを利用することで Enclave を実行したことを隠蔽できなくする仕組み，鍵発行処理を実行した形跡が KGC 以外のサーバに残る仕組みの導入が考えられる．後者に関しては，例えば Enclave 内での鍵発行を分散台帳上のスマートコントラクトを介して承認してからでなければ実行不可能とし，リクエストがブロックチェーンの台帳に必ず残るようにすることで，たとえ KGC 管理者でも鍵発行処理の実行を隠蔽できなくすることが考えられる．しかしながら，Enclave 内のプログラムがスマートコントラクトと通信を行う際には KGC を経由する必要があり，通信路上での改ざんや分散台帳のフォークを防ぐ手法が課題となる．また，仮にスマートコントラクトの機能制約によって安全な承認機能を持たせられない場合であっても，KGC 処理を分散し，各 KGC で相互に鍵発行処理のリクエストの記録を保存し合うようにすることで同様の効果を与えることが可能かもしれない．これらの考え方を組み合わせて，できるだけ負荷が少ないプロトコルに更新することは今後の課題とする．

7. まとめ

本稿では，北山らが提案した Intel SGX とブロックチェーンを用いた KGC に対して安全な ID ベース暗号方式について，鍵発行に関係する処理を実装し評価した．ペアリングライブラリ mcl を `MCL_USE_VINT` を有効にしてビルドすることで GMP 依存を排除し，Enclave 内でのペアリング演算を実現した．これにより，北山らの方式の初期化処理および鍵発行処理を Enclave 内に実装し，発行した個別秘密鍵の楕円エルガマル暗号による配布とマスター秘密鍵のシーリングによる保存を行った．さらに，実 SGX ハードウェア上で各処理の実行時間を Enclave 内外で比較計測した．計測の結果，従来の IBE に鍵供託問題への対策を付加した北山らの方式を Intel SGX 上で実装したところ，鍵発行処理に要する時間は約 3.24 ms であり，従来の IBE の約 0.49 ms と比べて約 6.6 倍となった．このオーバヘッドは Intel SGX の利用および不正防止のための署名・ログ生成の追加によるものであるが，鍵発行が利用開始時や再発行時にのみ行われることを踏まえれば，実用上許容できる範囲であることを示した．

今後の課題として，署名付き鍵生成ログをブロックチェーンに記録する処理の実装と，その際のガスコストおよび処理時間の計測，ならびにスマートコントラクトによる検証処理の自動化 [11] が挙げられる．また，malicious な攻撃者に対して安全なプロトコルの構成や，サイドチャネル攻撃への対策の検討も今後の課題である．

謝辞　本研究の一部は JSPS 科研費（課題番号 22K12034, 26K14822）の助成を受けたものである．

参考文献

[1] A. Shamir, “Identity-Based Cryptosystems and Signature Schemes,” *Advances in Cryptology – CRYPTO 1984*, LNCS 196, pp. 47–53, 1985.

[2] D. Boneh and M. Franklin, “Identity-Based Encryption from the Weil Pairing,” *Advances in Cryptology – CRYPTO 2001*, LNCS 2139, pp. 213–229, 2001.

[3] K. Emura, S. Katsumata, and Y. Watanabe, “Identity-Based Encryption with Security against the KGC: A Formal Model and Its Instantiation from Lattices,” *ESORICS 2019*, pp. 113–133, 2019.

[4] K. Emura, S. Katsumata, and Y. Watanabe, “Identity-Based Encryption with Security against the KGC: A Formal Model and Its Instantiations,” *Theoretical Computer Science*, vol. 900, pp. 97–119, 2022.

[5] S. Ema, Y. Sato, H. Nishino, K. Emura, and T. Ohigashi, “Implementation and Evaluation of an Identity-Based Encryption with Security against the KGC,” *Journal of Information Processing*, vol. 33, pp. 185–196, 2025.

[6] 北山武輝，佐藤龍，宮大介，渡邉英伸，柿崎淑郎，大東俊博，“Intel SGX とブロックチェーンを用いた KGC に対して安全な ID ベース暗号,” 電子情報通信学会技術研究報告, vol.125, no. 223, ICSS2025-72, pp.1-7, 2025.

[7] herumi, “mcl: a portable and fast pairing-based cryptography library,” `https://github.com/herumi/mcl`.

[8] S. Bowe, “BLS12-381: New zk-SNARK Elliptic Curve Construction,” Electric Coin Company, 2017.

[9] J. V. Bulck, M. Minkin, O. Weisse, *et al.*, “Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution,” *USENIX Security 2018*, pp. 991–1008, 2018.

[10] J. V. Bulck, D. Moghimi, M. Schwarz, *et al.*, “LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection,” *IEEE S&P 2020*, pp. 54–72, 2020.

[11] 佐藤龍，今村光良，大東俊博，“ブロックチェーンに基づく分散属性ベース暗号の Ethereum における実装および評価,” 情報処理学会研究報告，vol. 2020-CSEC-91，no. 31，pp. 1–8，2020.

© 2026 Information Processing Society of Japan